

Bubble API Usage

BlockPro Flutter App — Backend API Reference

Overview

Base URL	BUBBLE_API_BASE_URL env var → ../api/1.1/wf/
HTTP client	Dart http package
Auth method	Bearer token in Authorization header
Config file	lib/core/config/api_config.dart
HTTP helpers	lib/repositories/api_repository.dart

All authenticated requests include these headers:

```
Authorization: Bearer <token>
Content-Type: application/json
```

Endpoints

1. login — User Authentication

Path	login
Method	POST
Auth required	No
Source file	lib/repositories/auth_repository.dart

When called: User taps the login button on the login screen.

Request body:

```
{
  "email": "<email>",
  "password": "<password>"
}
```

Response:

```
{
  "status": "success",
  "response": {
    "token": "<bearer-token>",
    "user_id": "<user-id>",
    "expires": 86400
  }
}
```

What happens next: The bearer token is stored in-memory and persisted to SharedPreferences. Token expiry is calculated from the expires field. On app restart, the stored token is restored and checked for expiration.

2. fetchbuildings — Download Buildings

Path	fetchbuildings
Method	GET
Auth required	Yes
Source file	lib/repositories/sync_repository.dart

When called: Phase 1 of the initial sync after login. Also called on retry.

Response: JSON array of building objects.

```
[
  {
    "id": "abc123",
    "name": "Block A",
    "List of assets": ["asset1", "asset2"]
  }
]
```

Parsed fields:

JSON key	Model field	Notes
id or _id	Building.id	
name or Name	Building.name	Falls back to "Unnamed"
List of assets	Building.assetCount	Length of array

What happens next: Buildings are upserted into the local SQLite buildings table.

Response format notes: The parser handles three shapes: a direct JSON array, a map with a response key, or a JSON string needing a second decode.

3. fetchassets — Download Assets for a Building

Path	fetchassets
Method	GET
Auth required	Yes
Source file	lib/repositories/sync_repository.dart

When called: Phase 2 of initial sync — one call per building, sequentially.

Query params: block_id = The building ID

Response: JSON array of asset objects.

```
[
  {
    "assetId": "xyz789",
    "assetName": "Fire Extinguisher #1",
    "date_of_next_inspection": "2026-06-15",
    "date_of_previous_inspection": "2025-12-15",
    "interval_number_of_days": 182
  }
]
```

Parsed fields:

JSON key	Model field	Notes
assetId or id	Asset.id	
assetName or name	Asset.name	Falls back to "Unnamed"
date_of_next_inspection	Asset.nextInspection	Parsed as DateTime
date_of_previous_inspection	Asset.previousInspection	Parsed as DateTime
interval_number_of_days	Asset.intervalDays	Integer

What happens next: Assets are upserted into the local SQLite assets table, linked via buildingId.

4. fetchquestions — Download Question Templates

Path	fetchquestions
Method	GET
Auth required	Yes
Source file	lib/repositories/sync_repository.dart

When called: Not currently invoked during initial sync. The method exists but checklists (endpoint 5) are used instead.

Query params: asset_id = The asset ID

```
[
  {
    "question_id": "q001",
    "question_text": "Is the pressure gauge in the green zone?",
    "answer_option": "Yes/no",
    "photo_requirement": "Always"
  }
]
```

5. fetchchecklist — Download Checklist for an Asset

Path	fetchchecklist
Method	GET
Auth required	Yes
Source file	lib/repositories/sync_repository.dart

When called: (1) Initial sync Phase 3 — one call per asset, parallel with max concurrency of 5. (2) Background sync — triggered on inspection/asset detail screen load.

Query params: asset_id = The asset ID

Response: Nested envelope with a malformed checklist string.

```
{
  "status": "success",
  "response": {
    "checklist": "{\"question_id\":\"q1\",\"question_text\":\"...\"},..."
  }
}
```

Parsed fields:

JSON key	Model field	Notes
question_id / id / _id	Question.id	
question_text / questionText / Question text	Question.questionText	
answer_option / answerOption / Answer option	Question.answerOption	Enum: Yes/no, Satisfactory/unsatisfactory, Yes/no/n/a
photo_requirement / photoRequirement / Photo requirement	Question.photoRequirement	Enum: Always, Only when no/unsatisfactory

What happens next: Questions are upserted into the local SQLite questions table with `source = 'checklist'`.

Response format notes: The parser unwraps the envelope, replaces smart/curly quotes, wraps concatenated JSON objects in array brackets, and handles double-encoded strings.

6. upload-image — Upload Inspection Photo

Path	upload-image
Method	POST
Auth required	Yes
Source file	lib/providers/inspection_provider.dart

When called: During inspection submission — before the final payload is sent. Each photo uploaded individually.

Request body:

```
{
  "base64": "<base64-encoded-image-bytes>",
  "asset_id": "<asset-id>",
  "filename": "photo_2026-03-01.jpg"
}
```

Response:

```
{
  "response": {
    "image_id": "img_abc123"
  }
}
```

What happens next: The returned `image_id` is collected into a list. Failed uploads are logged but submission continues (best-effort). All IDs are included in the `completed-inspection` payload.

7. completed-inspection — Submit Completed Inspection

Path	completed-inspection
Method	POST
Auth required	Yes
Source file	lib/providers/inspection_provider.dart

When called: User taps "Submit" on the inspection screen, after all photos have been uploaded.

Request body:

```
{
  "asset_id": "<asset-id>",
  "answers": [
    {
      "question_text": "Is the pressure gauge in the green zone?",
      "answer_text": "Yes"
    }
  ],
  "photo_ids": ["img_abc123", "img_def456"]
}
```

photo_ids is only included when at least one photo was uploaded. answer_text is the selected label or an empty string.

Validation: Every question with answer options must have a selected answer. If a photo is required, at least one must be attached.

Data Sync Flow

Initial Sync (after login)

Orchestrated by `SyncRepository.syncAll()`, driven by `InitialSyncNotifier`.

```
Login
|
v
Phase 1: fetchbuildings --> upsert into buildings table
|
v
Phase 2: fetchassets      --> one call per building (sequential)
                           upsert into assets table
|
v
Phase 3: fetchchecklist  --> one call per asset (parallel, max 5)
                           upsert into questions table
```

The initial sync screen shows progress through each phase. A retry button is available if any phase fails.

Background Sync (on-demand)

When the user navigates to an asset detail or inspection screen, `checklistStreamProvider` fires a background call to `fetchchecklist` for that asset, refreshing local data without blocking the UI.

Data Storage

All synced data is stored locally in SQLite (via Drift ORM). The app reads from the local database for all UI display, using the API only to populate/refresh the local cache.

Connectivity & Offline Handling

Source: `lib/services/connectivity_service.dart`

Combines hardware connectivity checks (`connectivity_plus`) with API success/failure signals. Changes are debounced (1s) to prevent UI flicker. Tracks both network interface availability and API reachability.

Key Files

File	Purpose
lib/core/config/api_config.dart	Base URL configuration from .env
lib/repositories/api_repository.dart	HTTP client with authenticated GET/POST helpers
lib/repositories/auth_repository.dart	Login, token storage, session management
lib/repositories/sync_repository.dart	Orchestrates all data sync
lib/utils/api_parsers.dart	Response parsing & normalization
lib/providers/inspection_provider.dart	Photo upload & inspection submission
lib/providers/initial_sync_provider.dart	Initial sync UI state machine
lib/providers/checklist_provider.dart	Background checklist sync & streaming
lib/services/connectivity_service.dart	Online/offline detection
lib/models/building.dart	Building model & JSON parsing
lib/models/asset.dart	Asset model & JSON parsing
lib/models/question.dart	Question model, answer options & photo requirement enums